# ReGuide: From Test-Time Guidance to Self-Improving Diffusion Policies

**Tzu-Hsiang Lin, Srinivas Shakkottai, Dileep Kalathil, and P. R. Kumar**
Texas A&M University

**Abstract:** Behavior-cloned diffusion policies are expressive but remain vulnerable to covariate shift: small deviations from demonstrated states can compound into task failure. Existing methods address this either by expanding the training distribution through expert corrections or synthetic augmentation, or by steering a frozen policy at test time with guidance from a learned model. The former can be expensive or assumption-dependent, while the latter discards the corrected trajectories after execution. We introduce **ReGuide**, a self-improving framework that treats guided rollouts as reusable on-policy recovery data. ReGuide first uses **Phase-Conditioned Guidance (PCG)** to generate corrective rollouts: it constructs phase-specific latent targets, applies guidance only in the drifted-but-recoverable regime, and guides through the estimated clean action to match the dynamics model's training distribution. Successful guided rollouts are then absorbed back into the policy through **ReGuide-FT**, which fine-tunes the current checkpoint, or **ReGuide-FS**, which retrains from scratch on the augmented dataset; the two can also be composed and iterated. On Robomimic Can, Square, Transport, and Tool Hang, ReGuide improves base-policy success by $1.3$–$7.7\times$, outperforms LPB in the test-time-only setting, and matched-data ablations show that the gains come from guided recovery data rather than additional rollouts alone.

**Keywords:** Diffusion Policies, Test-Time Guidance, Imitation Learning

## 1 Introduction

Imitation learning-based policies are vulnerable to covariate shift: small errors move the robot away from demonstrated states, after which the policy is queried on observations outside its training distribution and errors compound over time [1]. This problem is especially acute for long-horizon manipulation, where an early deviation can make later subtasks unrecoverable. Diffusion policies improve the expressivity by modeling multimodal action distributions [2], but they do not remove this distribution-shift problem. A common solution is to expand the training distribution. DAgger-style methods collect expert corrections at learner-visited states [1, 3, 4], while data-augmentation methods synthesize additional examples using task structure, simulation, or local dynamics assumptions [5, 6, 7]. These methods address the right failure mode, but require either continued expert access or assumptions about which perturbations preserve task correctness. In contrast, recent test-time guidance methods such as DynaGuide [8] and Latent Policy Barrier (LPB) [9] steer a frozen diffusion policy using gradients from a learned dynamics model, improving recovery without retraining or new demonstrations. However, these methods discard the guided trajectory after execution.

In this paper, we propose a simple yet highly effective solution for this problem: **use guided rollouts as training data for iterative self-improvement**. A successful guided rollout captures precisely the information missing from the original demonstrations: where the learned policy tends to drift, and which corrective actions return the system toward task completion. Recycling such rollouts provides on-policy recovery data using only a trajectory-level success signal, rather than expensive per-state expert relabeling as used in Dagger-style algorithms.

Although this idea is simple to state, turning guided rollouts into useful training data requires several technical choices that prevent self-generated data from degrading the policy. Guided rollouts are useful only when the guidance signal is reliable. Long-horizon tasks are phase-structured, and a single global target set can pull the rollout toward the wrong stage or collapse valid behavior modes. Dynamics gradients are also reliable only near the data distribution: guidance applied too close to the expert manifold can perturb correct actions, while guidance applied too far away can rely on extrapolative predictions. Finally, standard diffusion guidance differentiates through noisy denoising iterates, whereas the dynamics model is typically trained on clean action chunks.

We introduce **ReGuide**, a self-improving diffusion-policy framework that turns this idea into a practical algorithm by controlling how guided rollouts are generated, filtered, and absorbed back into the policy. ReGuide first uses **Phase-Conditioned Guidance (PCG)** to generate reliable corrective rollouts: PCG constructs phase-specific latent targets from demonstrations, preserves multimodality through multiple targets per phase, gates guidance to the drifted-but-recoverable regime, and applies the guidance gradient through the estimated clean action following MPGD [10]. The successful guided rollouts are then recycled into policy training through two complementary update mechanisms: **ReGuide-FT**, which fine-tunes the current checkpoint with a rehearsal-style mixture of demonstrations and guided rollouts, and **ReGuide-FS**, which retrains a fresh policy on the augmented dataset. Crucially, the updated policy can be rolled out again under the same guidance mechanism to generate a new batch of higher-quality recovery data, yielding an iterative rollout–collect–train loop that improves the policy without additional expert demonstrations until the self-generated data reaches its performance ceiling.

On Robomimic tasks [11], ReGuide improves base-policy success by $1.3$–$7.7\times$. PCG outperforms LPB in the test-time-only setting, matched-data ablations show that guided rollouts are more valuable than unguided rollouts at the same data volume, and a second ReGuide-FT iteration further improves performance, showing that guided rollout generation can support iterative self-improvement.

## 2 Related Work

**Diffusion policies and test-time guidance.** We build on Diffusion Policy [2] as the base visuomotor policy. Recent work steers pretrained diffusion policies at inference time without modifying their weights: DynaGuide [8] uses gradients from an external dynamics model, Latent Policy Barrier (LPB) [9] treats expert latent embeddings as an implicit in-distribution barrier, PPGuide [12] learns a performance predictor for guidance, and Generative Predictive Control (GPC) [13] performs model-based look-ahead. These methods improve execution-time behavior, but the guided trajectories are discarded after each episode. ReGuide instead uses guidance as a data-generation mechanism: successful guided rollouts are added back to the training set to improve the policy. ReGuide also differs in the guidance design: we adapt Manifold Preserving Guided Diffusion [10] to differentiate through the estimated clean action, and replace global targets with phase-conditioned target sets and a two-threshold gate. LPB is the closest architectural baseline, since it also applies dynamics-gradient guidance to a diffusion policy in latent space.

**Self-improvement and data augmentation for imitation learning.** Covariate shift in behavior cloning is commonly addressed by expanding the learner's training distribution. Interactive methods such as DAgger [1] and RaC [3] collect new supervision at policy-visited states, but require continued expert access. Other approaches augment the data without further expert intervention [5, 6, 7, 14]. The closest in spirit is CCIL [5], which learns a Lipschitz-regularized dynamics model from demonstrations and generates corrective action labels near demonstrated states. ReGuide differs in both mechanism and loop structure: it uses test-time dynamics gradients to recover rollouts that have already drifted from the demonstrations, filters successful trajectories, and iteratively retrains or fine-tunes the policy on the resulting on-policy recovery data.

**Other related work.** Extended discussion of related work is provided in Appendix A.

## 3 Preliminaries

**Diffusion policy.** Given expert demonstrations $\mathcal{D}_{\text{expert}} = \{(o_t, a_t)_{t=1}^{T}\}$, where $o_t \in \mathcal{O}$ and $a_t \in \mathcal{A}$ denote the observation and action at time $t$, an imitation learning algorithm learns a policy $\pi_\theta : \mathcal{O} \to \mathcal{A}$ by minimizing a supervised loss. A policy is called a *diffusion policy* [2] when $\pi_\theta$ is parameterized as a conditional denoising diffusion model, conditioned on the past observations. Following Chi et al. [2], the policy conditions on a window of $T_o$ past observations $O_t = (o_{t-T_o+1}, \ldots, o_t)$ and generates a chunk of $T_p$ future actions $A_t = (a_t, a_{t+1}, \ldots, a_{t+T_p-1})$, of which the first few are executed before re-planning. In practice, the denoiser conditions on a low-dimensional latent embedding $z_t = h(O_t)$ rather than the raw observation window.

To sample an action chunk $A_t \sim \pi_\theta(\cdot \mid O_t)$, a diffusion policy starts from Gaussian noise $A_t^K \sim \mathcal{N}(0, I)$ and iteratively denoises through $K$ reverse-diffusion steps. At each step $k$, a learned noise predictor $\epsilon_\theta(A_t^k, k, z_t)$ produces the reverse update

$$A_t^{k-1} \;=\; \frac{1}{\sqrt{\alpha_k}}\left(A_t^k \;-\; \frac{1-\alpha_k}{\sqrt{1-\bar{\alpha}_k}}\,\epsilon_\theta(A_t^k, k, z_t)\right) \;+\; \sigma_k\,\xi, \qquad \xi \sim \mathcal{N}(0, I), \tag{1}$$

where $\alpha_k$, $\bar{\alpha}_k$, and $\sigma_k$ are determined by the noise schedule and $\xi$ is set to zero at the final step ($k = 1$). Iterating from $k = K$ down to $k = 1$ yields the executed action chunk $A_t = A_t^0$.

**Guidance for diffusion policies.** Diffusion models admit a natural mechanism for steering generation at inference time: the reverse process can be biased toward a desired property with the gradient of a differentiable reward function. Concretely, when generating a sample $x$ via reverse diffusion from $x_K \sim \mathcal{N}(0, I)$ down to $x_0$, classifier guidance [15] and its classifier-free variants [16] modify the predicted noise at step $k$ as

$$\hat{\epsilon}(x_k) \;=\; \epsilon_\theta(x_k) \;-\; \eta\sqrt{1-\bar{\alpha}_k}\,\nabla_{x_k} r(x_k), \tag{2}$$

where $r$ is a reward (e.g., a classifier log-probability for a target class, text-to-image fidelity for the generated image), and $\eta$ controls the guidance strength. The gradient $\nabla_{x_k} r(x_k)$ pushes each denoising step toward samples that score higher under $r$, so the final $x_0$ tends to satisfy the reward criterion while still lying on the data manifold learned by $\epsilon_\theta$. This principle has been used to steer image generation toward target classes, text descriptions, and aesthetic properties without retraining the underlying diffusion model.

Recent work transfers this idea from image generation to control policies represented as diffusion models. In offline reinforcement learning, several methods guide a diffusion policy using the gradient of a learned value function [17, 18, 19], steering action generation toward high-value behavior. In imitation learning, DynaGuide [8] and LPB [9] instead use a learned dynamics model to keep rollouts close to the expert distribution. The general form mirrors the image-generation case above: at denoising step $k$, the noise prediction is augmented with the gradient of a differentiable cost $\delta$,

$$\hat{\epsilon}(A_t^k) \;=\; \epsilon_\theta(A_t^k) \;-\; \eta\sqrt{1-\bar{\alpha}_k}\,\nabla_{A_t^k}\,\delta(A_t^k), \tag{3}$$

with $x_k \leftrightarrow A_t^k$ and the reward $r$ replaced by a cost $\delta$ that penalizes drift from the expert manifold. The intuition is the same as before: each denoising step is nudged toward actions whose predicted consequences score well under $\delta$.

**Dynamics model.** A dynamics model predicts how the environment evolves under a given action: given the latent state $z_t$ and a candidate action chunk $A_t$, it returns the latent state the environment is expected to reach, $\hat{z}_{t+T_p} = d_\phi(z_t,\ A_t)$,, where $z_t = h(o_t)$, and $h$ is the visual encoder shared with the diffusion policy. Following [9], we use the cost function $\delta(A_t^k) \;=\; \left\| d_\phi(z_t, A_t^k) - z^* \right\|^2$, where $z^*$ is a target latent state. The gradient $\nabla_{A_t^k}\delta$ then pushes each denoising step toward action chunks whose predicted consequences lie close to $z^*$, shifting the guidance signal from "does this action look like an expert action" to "does this action lead to an expert-like future state."

We adopt the DINO-WM architecture [20] for $d_\phi$, following [9]. The dynamics model shares its visual encoder $h$ with the diffusion policy so that latent-space distances are meaningful as a guidance signal, and is trained on rollouts (both successful and failed) generated by the diffusion policy.

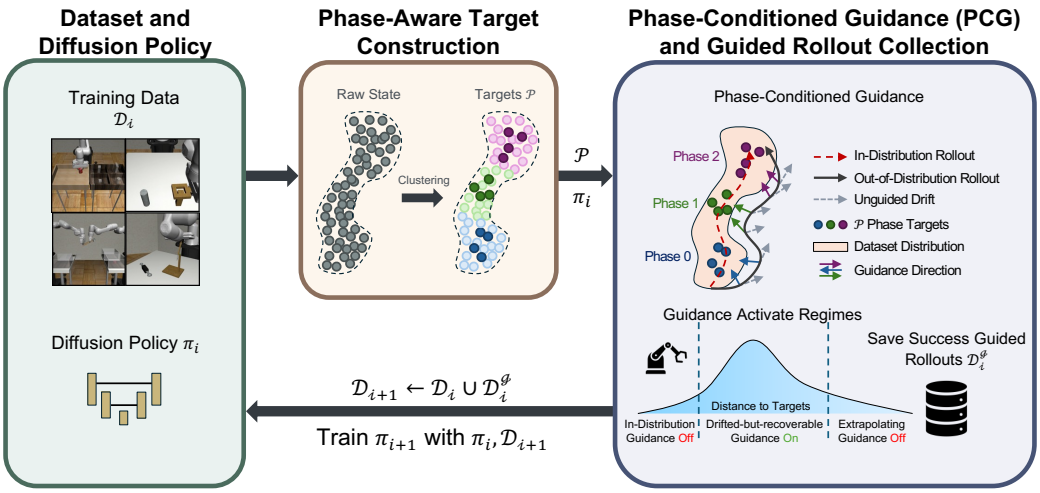


Figure 1: **ReGuide overview.** At iteration $i$: starting with $\pi_i$ and $\mathcal{D}_i$, construct phase targets $\mathcal{P}$ by clustering the latent states, roll out $\pi_i$ with phase-conditioned guidance (active only in the drifted-but-recoverable regime per the per-phase distance distribution), and merge successful guided rollouts $\mathcal{D}_i^g$ into $\mathcal{D}_{i+1}$ for the next iteration. Update the policy to get $\pi_{i+1}$ from $\mathcal{D}_{i+1}$ and $\pi_i$

## 4 Our Approach: ReGuide

ReGuide turns guidance into a data-generation mechanism for a behavior-cloned diffusion policy without collecting additional expert demonstrations. It consists of three coupled components. First, *phase-aware target construction* (Section 4.1) decomposes long-horizon demonstrations into temporally ordered phases and constructs a set of latent targets for each phase, so that "in-distribution" behavior is defined locally. Second, *phase-conditioned guidance and data collection* (Section 4.2) uses a learned dynamics model to guide the diffusion denoising process toward the appropriate phase targets, but only in regimes where the predicted future latent is drifted from the demonstrations while still within the model's reliable operating range. Third, *iterative self-improvement learning* (Section 4.3) filters successful guided rollouts, merges them with the existing dataset, and updates the policy through either fine-tuning or retraining. The updated policy then generates a new distribution of guided rollouts, yielding an iterative rollout–collect–train loop driven by self-generated, success-filtered data.

**Algorithm 1** ReGuide

**Require:** Demonstration data $\mathcal{D}_{\text{demo}}$, number of iterations $N$
1: $\mathcal{D}_1 \leftarrow \mathcal{D}_{\text{demo}}$
2: Train $\pi_1$ on $\mathcal{D}_1$
3: Construct phase targets and distance distributions via Alg. 3
4: **for** $i = 1$ to $N$ **do**
5: Collect successful rollouts $\mathcal{D}_i^g$ from $\pi_i$ via Alg. 2
6: $\mathcal{D}_{i+1} \leftarrow \mathcal{D}_i \cup \mathcal{D}_i^g$
7: Train $\pi_{i+1}$ using $\mathcal{D}_{i+1}$ and $\pi_i$
8: **end for**
9: **return** $\pi_{N+1}$

Algorithm 1 and Figure 1 summarize our ReGuide Algorithm.

### 4.1 Phase-Aware Target Construction

ReGuide requires guidance targets that are precise enough to correct deviations, but not so restrictive that they collapse valid modes of expert behavior. This is especially important in long-horizon manipulation, where visually similar observations may correspond to different temporal stages, and where each stage can contain multiple valid action modes. A single global expert latent set can therefore produce ambiguous guidance: it may pull a rollout toward the wrong stage of the task, or treat a valid mode as out-of-distribution. We address this by constructing phase-conditioned target sets from the current training data. Figure 2 gives the qualitative summary. Formal description is given in Algorithm 3, which is deferred to the Appendix due to page constraint.

For each state, we form an augmented feature vector $[v_t,\ p_t,\ \Delta v_t,\ \Delta p_t]$, where $v_t$ and $p_t$ denote visual and proprioceptive latents, and $\Delta v_t$, $\Delta p_t$ are one-step temporal differences. The temporal differences help disambiguate states that are visually similar but belong to different phases of the task. We cluster

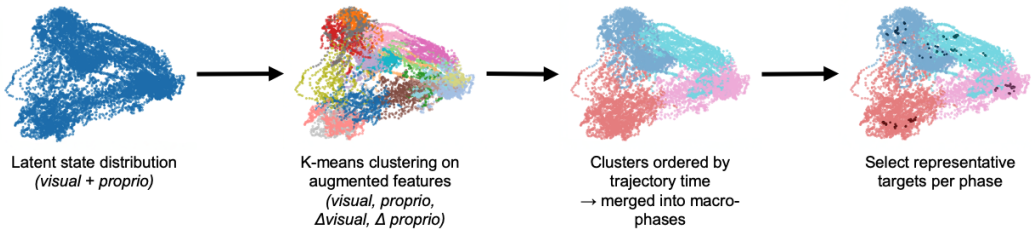


Figure 2: **Phase-aware target construction.** Latents states are clustered using temporally augmented features, ordered by trajectory time, and grouped into macro-phases. Representative centroids from each phase define the target sets used for phase-conditioned guidance.

these features after dimensionality reduction using a standard Principal Component Analysis (PCA), sort clusters by their mean timestep, and merge adjacent clusters into $N_p$ macro-phases. For each phase $j$, we select $M$ representative cluster centroids as the phase target set $\mathcal{P}_j = \{p_{j,1}, \ldots, p_{j,M}\}$. Given a predicted latent $z$, its distance to phase $j$ is defined by a soft minimum over phase targets, $\mathcal{L}(z, \mathcal{P}_j) = -\tau \log \sum_{m=1}^{M} \exp\left(-\frac{|z-p_{j,m}|^2}{\tau}\right)$, where $\tau$ controls the sharpness of the minimum. This preserves multimodality within each phase while providing a differentiable guidance objective. We also store the empirical distance distribution $\mathcal{F}_j = \mathcal{L}(z_t, \mathcal{P}_j) : z_t$ belongs to phase $j$. These per-phase distributions are used in Section 4.2 to calibrate the guidance, so that the decision to guide depends on the local geometry of the current phase rather than on a global distance threshold.

### 4.2 Phase-Conditioned Guidance (PCG) and Rollout Data Collection

Given the phase target sets from Section 4.1, ReGuide guides the diffusion policy during rollout generation. The goal is not only to improve the current episode, but to produce trajectories that can be reused for training. For this, the guidance must be applied in a form compatible with the learned dynamics model, and only in regions where the model provides reliable gradients.

**Algorithm 2** Phase-Conditioned Guidance and Rollout Collection.

**Require:** Number of macro-phases $N_p$, Phase targets $\{\mathcal{P}_j\}_{j=1}^{N_p}$, guided denoising steps $K_g$, thresholds $\{(\ell_{\text{low}}^{(j)}, \ell_{\text{high}}^{(j)})\}_{j=1}^{N_p}$.

$j \leftarrow 1$
**for** $t = 0$ to $T-1$ **do**
  Sample $A_t^K \sim \mathcal{N}(0, I)$
  **for** $k = K$ down to 1 **do**
    $\hat{\epsilon} \leftarrow \epsilon_\theta(A_t^k, k)$
    $\hat{A}_t^0 \leftarrow$ Eq. (4). $\ell \leftarrow \mathcal{L}(d_\phi(z_t, \hat{A}_t^0), \mathcal{P}_j)$
    **if** $k \le K_g$ **and** $\ell_{\text{low}}^{(j)} < \ell < \ell_{\text{high}}^{(j)}$ **then**
      Update $\hat{\epsilon}$ via Eqs. (5)–(6)
    **end if**
    $A_t^{k-1} \leftarrow$ Eq. (1) with $\hat{\epsilon}$
  **end for**
  Guidance action $A_t^g \leftarrow A_t^0$. Execute $A_t^g$
  Update $j$ per consecutive rule (Sec. 4.2)
**end for**
**return** $\mathcal{D}^g = \{(O_t, A_t^g)\}_{t=0}^{T-1}$

Prior diffusion-policy guidance methods typically differentiate the guidance objective with respect to the noisy denoising iterate $A_t^k$. This is a mismatch for our dynamics model $d_\phi$, which is trained on clean action chunks. We therefore adapt the MPGD-style update [10] and apply the guidance gradient through the estimated clean action. At denoising step $k$, we first compute $\hat{A}_t^0$ via Eq. (4). For the current phase $j$, the dynamics model predicts the future latent $d_\phi(z_t, \hat{A}_t^0)$, and the phase-conditioned guidance objective is the soft-minimum distance from this latent to $\mathcal{P}_j$. We then update the estimated clean action via Eq. (5). Since the diffusion scheduler expects a noise prediction, we convert the guided clean-action estimate back into the corresponding guided noise via Eq. (6).

Guidance is useful only when the rollout has deviated from the demonstrations but remains recoverable. If the predicted future latent is already close to the current phase targets, guidance is unnecessary and can perturb a correct action. If it is too far from the phase distribution, the dynamics model is extrapolating, and its

$$\hat{A}_t^0 = \frac{A_t^k - \sqrt{1-\bar{\alpha}_k}\,\epsilon_\theta}{\sqrt{\bar{\alpha}_k}}, \tag{4}$$

$$\tilde{A}_t^0 \leftarrow \hat{A}_t^0 - \eta\sqrt{1-\bar{\alpha}_k}\,\nabla_{\hat{A}_t^0}\,\mathcal{L}\big(d_\phi(z_t,\ \hat{A}_t^0),\ \mathcal{P}_j\big), \tag{5}$$

$$\hat{\epsilon} = \frac{A_t^k - \sqrt{\bar{\alpha}_k}\,\tilde{A}_t^0}{\sqrt{1-\bar{\alpha}_k}}. \tag{6}$$

gradient may generate low-quality actions. ReGuide, therefore, uses a two-threshold gate for each phase. Let $\ell = \mathcal{L}\big(d_\phi(z_t,\ \hat{A}_t^0), \mathcal{P}_j\big)$ be the phase-conditioned distance at the current denoising step. From the empirical distance distribution $\mathcal{F}_j$, we define lower and upper thresholds $\ell_{\text{low}}^{(j)}$ and $\ell_{\text{high}}^{(j)}$ as fixed percentiles. Guidance is applied only when $\ell_{\text{low}}^{(j)} < \ell < \ell_{\text{high}}^{(j)}$. Thus, the lower threshold avoids unnecessary intervention near the demonstration manifold, while the upper threshold prevents the method from trusting dynamics gradients in extrapolation regions. Because the thresholds are phase-specific, the gate adapts to the local geometry of each stage of the task.

During execution, ReGuide maintains the current phase index $j$. We advance from phase $j$ to $j+1$ when the predicted latent is closer to $\mathcal{P}_{j+1}$ than to $\mathcal{P}_j$ by a margin $\Delta$ for $K$ consecutive rollout steps, which prevents transient prediction noise from causing premature phase switches. After rollout completion, we retain only successful guided trajectories. Each retained trajectory is stored as $\{(O_t, A_t^g)\}_{t=0}^{T-1}$, where $A_t^g$ is the guided action chunk at step $t$. The trajectory is added to the guided buffer $\mathcal{D}_i^g$, which is merged into the training set for the next ReGuide iteration: $\mathcal{D}_{i+1} \leftarrow \mathcal{D}_i \cup \mathcal{D}_i^g$.

### 4.3 Iterative Self-Improvement Learning

The rollout procedure in Algorithm 2 converts guidance into Dagger-style training data: at iteration $i$, we run the current policy with phase-conditioned guidance, keep only successful trajectories, and merge them into a guided buffer $\mathcal{D}_i^g$. The updated policy then collects the next batch of guided rollouts, forming a rollout–collect–train loop that can be run for a single round or repeated across multiple iterations; we reuse the dynamics model trained on the base policy's rollouts across all iterations rather than retraining it. We consider three ways to absorb the guided data into the policy.

**ReGuide-FT.** The first variant fine-tunes the current policy checkpoint on a rehearsal buffer containing both prior training data and newly collected guided rollouts. At iteration $i$, minibatches are sampled from the existing dataset $\mathcal{D}_i$ and the guided buffer $\mathcal{D}_i^g$ with a fixed ratio $\rho$. This update is computationally efficient and preserves the competence of the current policy while exposing it to recoverable states reached during guided execution.

**ReGuide-FS.** The second variant trains a fresh diffusion policy from scratch on the augmented dataset $\mathcal{D}_i \cup \mathcal{D}_i^g$. This removes dependence on the previous checkpoint and can yield a different solution when the base policy is a poor initialization, at the cost of a full retraining run.

**ReGuide-FS→FT.** Since ReGuide-FT operates on any starting checkpoint, we can stack the variants: apply ReGuide-FT on top of a ReGuide-FS policy rather than the base. ReGuide-FS provides a stronger initialization than the base, so the FT round refines an already-competent policy; the composition costs both runs but yields ReGuide's best results on Can, Square, and Transport in Table 1.

## 5 Experiments

**Experimental setup.** We evaluate ReGuide on four Robomimic manipulation tasks [11]: Can, Square, Transport, and Tool Hang. For each task, we train a diffusion policy on a small subset of the available demonstration data: 15 demonstrations for Can, 30 for Square, 10 for Transport, and 80 for Tool Hang. This places the base policies in a low-data regime where behavior cloning is susceptible to covariate shift. We train a latent-space dynamics model using rollouts from the corresponding base policy, and construct phase targets and per-phase distance distributions from the current training data. Each reported success rate is computed from 2,500 rollouts of a fixed trained checkpoint, using 50 initial seeds and 50 rollouts per seed. We report the mean success rate $\pm$ standard error of the mean. Additional implementation details are given in Appendix D.

**Main results.** Figure 3 and Table 1 summarize the main results. All three ReGuide variants improve substantially over the base diffusion policy. **ReGuide-FT**, fine-tuning from the base checkpoint, lifts success by 1.2–1.5$\times$ on Can, Square, and Transport, and by 7.7$\times$ on Tool Hang after two iterations. **ReGuide-FS**, retraining from scratch on demonstrations plus guided rollouts, achieves 1.3–1.5$\times$ on the same three tasks and 5.3$\times$ on Tool Hang. Neither single-variant configuration uniformly

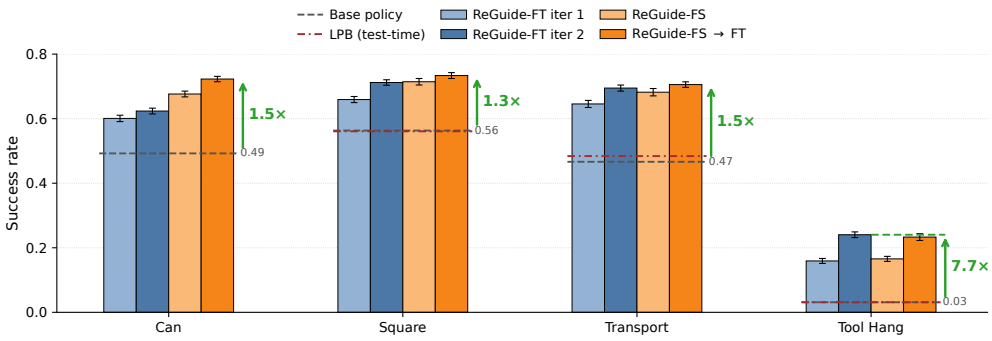


Figure 3: **Main results.** ReGuide improves over the base diffusion policy across all tasks. ReGuide-FT and ReGuide-FS are complementary variants; their composition ReGuide-FS→FT gives the best result on Can, Square, and Transport, while iterated ReGuide-FT remains slightly stronger on Tool Hang.

Table 1: Success rates across Robomimic tasks. Our approaches, **PCG**, **ReGuide-FT**, **ReGuide-FS**, and **ReGuide-FS→FT**, all outperform the base policy and LPB. The composition ReGuide-FS→FT achieves the best result on Can, Square, and Transport; iterated ReGuide-FT remains slightly stronger on Tool Hang. For each task, the highest success rate is in **bold** and the second-best is underlined.

| | Test-Time Guidance | | | ReGuide-FT | | | |
|---|---|---|---|---|---|---|---|
| Task | Base Policy | LPB [9] | PCG | Iteration 1 | Iteration 2 | ReGuide-FS | ReGuide-FS → FT |
| Can | 0.4924 ± 0.0101 | — | 0.5012 ± 0.0100 | 0.6008 ± 0.0098 | 0.6236 ± 0.0090 | 0.6764 ± 0.0089 | **0.7228** ± 0.0083 |
| Square | 0.5632 ± 0.0091 | 0.5608 ± 0.0099 | 0.5836 ± 0.0104 | 0.6592 ± 0.0095 | 0.712 ± 0.0085 | 0.7144 ± 0.0101 | **0.7336** ± 0.0091 |
| Transport | 0.4664 ± 0.0086 | 0.4840 ± 0.0082 | 0.5140 ± 0.0102 | 0.6456 ± 0.0110 | 0.6948 ± 0.0094 | 0.6820 ± 0.0114 | **0.7056** ± 0.0086 |
| Tool Hang | 0.0312 ± 0.0030 | 0.0312 ± 0.0036 | 0.0384 ± 0.0036 | 0.1592 ± 0.0077 | **0.2404** ± 0.0088 | 0.1656 ± 0.0078 | 0.2332 ± 0.0106 |

dominates: ReGuide-FS leads on Can, the two are comparable on Square, and ReGuide-FT's second iteration slightly exceeds ReGuide-FS on both Transport and Tool Hang. **ReGuide-FS→FT**, the composition, achieves the best lift on Can, Square, and Transport ($1.5\times$, $1.3\times$, $1.5\times$), confirming that the two variants are complementary: ReGuide-FS moves the policy to a stronger starting checkpoint, while ReGuide-FT efficiently refines it. On Tool Hang, however, the composition ($7.5\times$) lands slightly below ReGuide-FT's second iteration ($7.7\times$). We attribute this to the base policy being weak enough ($3\%$ success) that ReGuide-FS cannot reach a meaningfully stronger starting checkpoint within the limited training budget, so iterating FT extracts more signal from successive guided rollout batches than swapping checkpoints does.

**Iterative self-improvement.** Figure 4 shows the success rate as a function of the number of guided rollouts collected for updating the policy for iteration 1 and iteration 2 of ReGuide-FT. Our goal is to evaluate whether the rollout–collect–train loop continues to improve after one round. Starting from the best first-iteration ReGuide-FT policy, we collect a new batch of guided rollouts and perform a second ReGuide-FT iteration. The second iteration outperforms the first on all four tasks: ReGuide-FT's lift over the base grows from $1.2\times$ to $1.3\times$ on Can, $1.2\times$ to $1.3\times$ on Square, $1.4\times$ to $1.5\times$ on Transport, and $5.1\times$ to $7.7\times$ on Tool Hang. The gains are not unbounded: reaching the second-iteration peak requires more total guided rollouts, suggesting diminishing returns as the policy improves and the fixed dynamics model becomes less aligned with the updated policy.

**Comparison between LPB and PCG.** We compare our PCG apporach to LPB [9] in the test-time-only setting (no finetuning) in Table 1. PCG wins on every task where LPB is evaluated. These test-time gains are much smaller than the full ReGuide gains in Table 1, which is expected: the main role of PCG is not to improve a single rollout but to generate cleaner data for the subsequent ReGuide-FT and ReGuide-FS updates, where the $1.3$–$7.7\times$ lifts come from. Other guidance methods (DynaGuide [8], PPGuide [12], GPC [13]) use different guidance objectives or supervision assumptions, while data-augmentation methods such as CCIL [5] operate outside the inference-time guidance setting; we therefore treat LPB as the primary apples-to-apples comparison.

**Ablation studies.** The appendix isolates the main algorithmic choices in ReGuide:

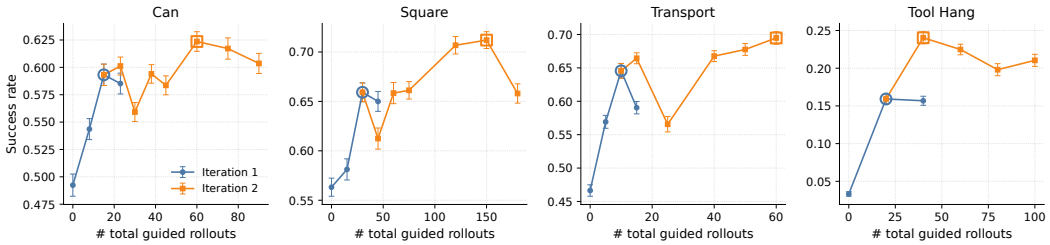


Figure 4: **Iterative self-improvement.** The second iteration of ReGuide-FT improves over the first on all tasks, showing that updated policies can generate useful new guided rollouts. The x-axis shows the cumulative number of guided rollouts collected to update the policy.

1. **Number of phase targets.** Table 2a sweeps the number of targets per phase $M$ on Transport. A single target ($M = 1$) does not improve over the base policy and very large $M$ also underperforms; an intermediate $M \approx 50$ is best, supporting the need to preserve multimodality without diluting the guidance signal.
2. **Two-threshold guidance gate.** Table 2b compares gating rules. A lower-only gate (matching prior work) lifts success by $+0.014$ over no guidance, while the full lower-and-upper gate lifts by $+0.044$—the upper threshold accounts for most of the benefit, supporting the three-regime design (guide only when drifted but recoverable).
3. **Clean-action guidance.** Table 3 compares differentiating the guidance objective through the noisy iterate $A_t^k$ versus the estimated clean action $\hat{A}_t^0$. Clean-action guidance wins by $\approx 2$ SE on Transport, consistent with the dynamics model being trained on clean action chunks.
4. **Guidance vs. additional data.** Tables 6 and 4 (with per-task detail in Tables 6a–7) compare guided rollouts with unguided base-policy rollouts at matched data volume. Guided rollouts outperform unguided rollouts under both ReGuide-FT and ReGuide-FS, showing ReGuide's gains are not merely from adding more rollouts.
5. **ReGuide-FT vs. ReGuide-FS.** Table 5 compares the two absorption mechanisms after the composition step on Can. The two are statistically comparable at matched rollout counts, while ReGuide-FT is more compute-efficient from an already-strong checkpoint.
6. **Per-task rollout-count sweeps.** Tables 8 and 9 give the per-task sweeps behind ReGuide-FS and the two-iteration ReGuide-FT curves in Figure 4, showing where each task saturates and the rollout-count vs. marginal-gain tradeoff.

## 6 Conclusion and Limitations

ReGuide turns test-time guidance from a one-time inference correction into reusable on-policy recovery data. A successful guided rollout captures where a behavior-cloned policy drifts and which actions recover task progress. ReGuide makes this usable for training through Phase-Conditioned Guidance, which localizes targets to task phases, gates guidance to the drifted-but-recoverable regime, and applies gradients through the estimated clean action. The successful rollouts are then absorbed through ReGuide-FT, ReGuide-FS, or their composition. Across Robomimic tasks, ReGuide improves base-policy success by $1.3$–$7.7\times$, outperforms LPB in the test-time-only setting, and continues improving across rollout–collect–train iterations.

**Limitations.** ReGuide has several limitations, which lead to natural extensions and future works. First, the dynamics model is reused across iterations for efficiency, but refreshing or uncertainty-calibrating it may improve later rounds. Second, rollout selection uses trajectory-level success filtering and random sampling; diversity-aware or quality-aware selection could improve data efficiency. Third, phase construction and gating thresholds are calibrated per task, suggesting future work on adaptive

phase discovery and threshold updates. Finally, our evaluation uses fixed checkpoints and simulated Robomimic tasks; broader training-seed studies and real-robot validation are important next steps.

## References


[1] S. Ross, G. Gordon, and D. Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS)*, volume 15 of *Proceedings of Machine Learning Research*, pages 627–635, 2011.

[2] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. Burchfiel, and S. Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Proceedings of Robotics: Science and Systems (RSS)*, 2023. doi:10.15607/RSS.2023.XIX.026.

[3] Z. Hu, R. Wu, N. Enock, J. Li, R. Kadakia, Z. Erickson, and A. Kumar. Rac: Robot learning for long-horizon tasks by scaling recovery and correction, 2025. URL https://arxiv.org/abs/2509.07953.

[4] X. Xu, Y. Hou, C. Xin, Z. Liu, and S. Song. Compliant residual dagger: Improving real-world contact-rich manipulation with human corrections, 2025. URL https://arxiv.org/abs/2506.16685.

[5] L. Ke, Y. Zhang, A. Deshpande, S. Srinivasa, and A. Gupta. CCIL: Continuity-based data augmentation for corrective imitation learning. In *International Conference on Learning Representations (ICLR)*, 2024.

[6] L. Ankile, A. Simeonov, I. Shenfeld, and P. Agrawal. Juicer: Data-efficient imitation learning for robotic assembly. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 5096–5103, 2024. doi:10.1109/IROS58592.2024.10802498.

[7] M. Jia, D. Wang, G. Su, D. Klee, X. Zhu, R. Walters, and R. Platt. SEIL: Simulation-augmented equivariant imitation learning. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pages 1845–1851, 2023.

[8] M. Du and S. Song. Dynaguide: Steering diffusion polices with active dynamic guidance, 2025. URL https://arxiv.org/abs/2506.13922.

[9] Z. Sun and S. Song. Latent policy barrier: Learning robust visuomotor policies by staying in-distribution, 2025. URL https://arxiv.org/abs/2508.05941.

[10] Y. He, N. Murata, C.-H. Lai, Y. Takida, T. Uesaka, D. Kim, W.-H. Liao, Y. Mitsufuji, J. Z. Kolter, R. Salakhutdinov, and S. Ermon. Manifold preserving guided diffusion. In *International Conference on Learning Representations (ICLR)*, 2024.

[11] A. Mandlekar, D. Xu, J. Wong, S. Nasiriany, C. Wang, R. Kulkarni, L. Fei-Fei, S. Savarese, Y. Zhu, and R. Martín-Martín. What matters in learning from offline human demonstrations for robot manipulation. In *Proceedings of the 5th Conference on Robot Learning (CoRL)*, volume 164 of *Proceedings of Machine Learning Research*, pages 1678–1690, 2022.

[12] Z. Wang, D. K. Jha, A. H. Qureshi, and D. Romeres. Ppguide: Steering diffusion policies with performance predictive guidance, 2026. URL https://arxiv.org/abs/2603.10980.

[13] H. Qi, H. Yin, A. Zhu, Y. Du, and H. Yang. Inference-time enhancement of generative robot policies via predictive world modeling, 2026. URL https://arxiv.org/abs/2502.00622.

[14] S. A. Mehta, Y. U. Ciftci, B. Ramachandran, S. Bansal, and D. P. Losey. Stable-bc: Controlling covariate shift with stable behavior cloning. *IEEE Robotics and Automation Letters*, 10(2): 1952–1959, 2025. doi:10.1109/LRA.2025.3526439.

[15] P. Dhariwal and A. Nichol. Diffusion models beat GANs on image synthesis. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 34, pages 8780–8794, 2021.

[16] J. Ho and T. Salimans. Classifier-free diffusion guidance. In *NeurIPS Workshop on Deep Generative Models and Downstream Applications*, 2021.

[17] K. Frans, S. Park, P. Abbeel, and S. Levine. Diffusion guidance is a controllable policy improvement operator, 2025. URL https://arxiv.org/abs/2505.23458.

[18] C. Lu, H. Chen, J. Chen, H. Su, C. Li, and J. Zhu. Contrastive energy prediction for exact energy-guided diffusion sampling in offline reinforcement learning. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, volume 202 of *Proceedings of Machine Learning Research*, pages 22825–22855, 2023.

[19] P. Hansen-Estruch, I. Kostrikov, M. Janner, J. G. Kuba, and S. Levine. Idql: Implicit q-learning as an actor-critic method with diffusion policies, 2023. URL https://arxiv.org/abs/2304.10573.

[20] G. Zhou, H. Pan, Y. LeCun, and L. Pinto. DINO-WM: World models on pre-trained visual features enable zero-shot planning. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267 of *Proceedings of Machine Learning Research*, pages 79115–79135, 2025.

[21] A. Mandlekar, F. Ramos, B. Boots, S. Savarese, L. Fei-Fei, A. Garg, and D. Fox. IRIS: Implicit reinforcement without interaction at scale for learning control from offline robot manipulation data. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pages 4414–4420, 2020.

[22] A. Gupta, V. Kumar, C. Lynch, S. Levine, and K. Hausman. Relay policy learning: Solving long-horizon tasks via imitation and reinforcement learning. In *Proceedings of the Conference on Robot Learning (CoRL)*, volume 100 of *Proceedings of Machine Learning Research*, pages 1025–1037, 2020.

[23] M. Shridhar, L. Manuelli, and D. Fox. Perceiver-actor: A multi-task transformer for robotic manipulation. In *Proceedings of the Conference on Robot Learning (CoRL)*, volume 205 of *Proceedings of Machine Learning Research*, pages 785–799, 2022.

[24] A. Mandlekar, S. Nasiriany, B. Wen, I. Akinola, Y. Narang, L. Fan, Y. Zhu, and D. Fox. MimicGen: A data generation system for scalable robot learning using human demonstrations. In *Proceedings of the Conference on Robot Learning (CoRL)*, volume 229 of *Proceedings of Machine Learning Research*, pages 1820–1864, 2023.

[25] Q. Xie, M.-T. Luong, E. Hovy, and Q. V. Le. Self-training with noisy student improves imagenet classification. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2020.

[26] S.-A. Rebuffi, A. Kolesnikov, G. Sperl, and C. H. Lampert. icarl: Incremental classifier and representation learning. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.

[27] P. Buzzega, M. Boschini, A. Porrello, D. Abati, and S. Calderara. Dark experience for general continual learning: a strong, simple baseline. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

## A Extended Related Work

This section expands on the related work discussion in Section 2, providing additional detail on individual methods and covering directions that did not fit in the main paper.

**Test-time guidance for diffusion policies.** DynaGuide [8] uses an external dynamics model to compute gradients that bias the denoising process toward a user-specified goal state, with guidance applied to the noisy denoising iterate at each reverse-diffusion step. Latent Policy Barrier (LPB) [9] adapts the same gradient mechanism to keep rollouts close to the expert distribution, treating expert latent embeddings as an implicit in-distribution barrier rather than as a goal to reach. PPGuide [12] replaces the dynamics-gradient signal with a learned performance predictor, steering toward action chunks that score well under the predictor. Generative Predictive Control (GPC) [13] takes a different approach: rather than gradient-based guidance, it samples multiple action candidates and uses an action-conditioned world model to rank them via lightweight model-based look-ahead. PCG differs from these methods along three axes: (i) it differentiates through the dynamics model with respect to the estimated clean action rather than the noisy iterate, matching the dynamics model's training distribution; (ii) it replaces a single global target set with phase-conditioned target sets that preserve expert multimodality within each phase; and (iii) it gates guidance to the drifted-but-recoverable regime via a two-threshold rule calibrated to per-phase distance distributions.

**Data augmentation for imitation learning.** Beyond CCIL discussed in the main paper, several methods expand the training distribution offline using task structure or assumptions about which perturbations preserve correctness. JUICER [6] combines expressive architectures with dataset-expansion and simulation-based augmentation for long-horizon assembly. SEIL [7] supplements expert trajectories with simulated transitions and exploits manipulation symmetries to generate equivalent demonstrations. Stable-BC [14] regularizes the policy so that its closed-loop behavior is locally stable around expert states. These methods share with ReGuide the goal of expanding training coverage without additional expert supervision, but rely on assumption-driven synthesis: known symmetries, local dynamics models, or stability conditions. ReGuide instead generates corrective data *on-policy* by rolling out the current policy under guidance and keeping successful trajectories, requiring only a trajectory-level success signal rather than assumptions about which perturbations preserve task correctness.

**Phase / sub-task decomposition in manipulation.** Decomposing long-horizon manipulation into temporally ordered phases or sub-skills is a recurring theme in robot learning, instantiated at several different levels of the pipeline. IRIS [21] factors the control problem into a high-level goal planner and a low-level goal-conditioned controller, learning sub-policies from offline demonstrations. Relay Policy Learning [22] learns goal-conditioned hierarchical policies from unsegmented demonstrations via a data-relabeling trick and then fine-tunes them with reinforcement learning. PerAct [23] sidesteps dense trajectory prediction altogether by predicting discrete keyframe end-effector poses that serve as phase anchors, with a motion planner handling the low-level control between them. MimicGen [24] uses an object-centric subtask decomposition for offline data generation, transforming and stitching demonstration segments to synthesize new trajectories. JUICER [6] similarly leverages subtask structure for both architecture design and augmentation in long-horizon assembly. ReGuide uses phases differently from all of these: the diffusion policy itself stays unfactored, the action representation stays continuous, and the demonstration data is not segmented for retargeting. Phases here serve only as an inference-time tool for localizing the "what counts as in-distribution" check that gates our guidance, adapting the threshold to local task geometry without imposing a structural decomposition on the policy, the data, or the action space.

**Iterative self-improvement and rehearsal-based learning.** Our rollout–collect–train loop is closer in spirit to self-training in supervised learning [25], where a model labels new data, filters by confidence, and retrains, than to classical continual learning, even though we borrow the rehearsal buffer construction from the latter. Rehearsal-based continual learning methods retain or replay a subset of past data during new updates to mitigate forgetting [26, 27]; our two-buffer formulation in Section 4.3 reuses this trick, with the buffer-share ratio $\rho$ functioning as a rehearsal fraction. The setting differs, though: continual learning typically targets sequential acquisition of *distinct* tasks, while we iteratively refine a policy on the *same* task using progressively higher-quality self-collected data. The closest neighbors are therefore self-training methods that filter generated data by an external

success signal, with ReGuide's distinguishing element being that the data-generation step is itself a guided diffusion rollout rather than a forward pass of the current model.

# B Algorithm

**Algorithm 3** Phase-Aware Target Construction

**Require:** Training data $\mathcal{D}_{\text{training}} = \{(o_t, a_t)\}$, visual encoder $E_v$, proprio encoder $E_p$, number of clusters $k$, number of macro-phases $N_p$, targets per phase $M$, PCA dimension $d$

**Ensure:** Per-phase phase target sets $\{\mathcal{P}_j\}_{j=1}^{N_p}$ and distance distributions $\{\mathcal{F}_j\}_{j=1}^{N_p}$

**Encode states with temporal augmentation:**
**for** each $(o_t, a_t) \in \mathcal{D}_{\text{training}}$ **do**
  $v_t \leftarrow E_v(o_t), \;\; p_t \leftarrow E_p(o_t)$ ▷ visual and proprio features
  $\Delta v_t \leftarrow v_t - v_{t-1}, \;\; \Delta p_t \leftarrow p_t - p_{t-1}$ ▷ zero at $t = 0$
  $z_t \leftarrow [v_t,\ p_t,\ \Delta v_t,\ \Delta p_t]$
**end for**
$\{\tilde{z}_t\} \leftarrow \text{PCA}(\{z_t\}, d)$
$\{c_t\} \leftarrow \text{KMeans}(\{\tilde{z}_t\}, k)$
**Order clusters temporally:**
**for** each cluster $c \in \{1, \ldots, k\}$ **do**
  $\bar{t}_c \leftarrow \text{mean}\{t : c_t = c\}$
**end for**
Sort clusters by $\bar{t}_c$ in ascending order
**Merge into macro-phases:** Group the $k$ ordered clusters into $N_p$ contiguous macro-phases $\{\Phi_1, \ldots, \Phi_{N_P}\}$
**Select phase targets and build distance distributions:**
**for** $j = 1$ to $N_p$ **do**
  $\mathcal{P}_j \leftarrow$ select $M$ representative latents from $\Phi_j$ (cluster centroids within $\Phi_j$)
  $\mathcal{F}_j \leftarrow \{\mathcal{L}(z_t, \mathcal{P}_j) : t \in \Phi_j\}$
**end for**
**return** $\{\mathcal{P}_j\}_{j=1}^{N_p}, \{\mathcal{F}_j\}_{j=1}^{N_p}$

# C Ablation Studies

This appendix collects all ablation studies referenced from the main paper:

- **PCG design choices** (below): number of targets per phase (Table 2a), two-threshold gating rule (Table 2b), and guidance target—clean action vs. noisy iterate (Table 3). All on Transport.
- **Guidance vs. additional data** (Section C.1): matched-data comparison of guided vs. unguided rollouts under both ReGuide-FT and ReGuide-FS, across Can, Square, and Transport.
- **ReGuide-FT vs. ReGuide-FS** (Section C.2): the two variants applied on top of the same checkpoint at matched rollout counts on Can.
- **Per-task rollout-count sweeps**: ReGuide-FS per task (Section C.3), the composition across tasks, and two-iteration ReGuide-FT.
- **Buffer-share ratio** (Table 10): sensitivity of ReGuide-FT to $\rho$ on Can.

**Number of Phase targets** In Section 4.1, we construct multiple targets per phase to respect the multimodal nature of expert manipulation behavior. Table 2a evaluates the effect of this choice on Transport. Performance is non-monotone in $M$: $M = 1$ (one target per phase) and very large $M$ both underperform; $M = 50$ achieves the best success rate.

Intuitively, too few phase targets over-commit the policy to whichever single modes happen to be captured, while too many dilute the gradient signal across loosely-related targets; the middle range balances these effects.

**Guidance Threshold** Table 2b compares two gating strategies for test-time guidance: applying guidance whenever the soft-minimum distance exceeds a lower threshold (the standard approach in prior work [9]), versus our two-threshold rule that additionally disables guidance when the distance exceeds an upper threshold. With both thresholds, guidance improves over the no-guidance baseline by $+0.044$. With only the lower threshold, the improvement shrinks to $+0.014$—essentially at baseline. The upper threshold accounts for the majority of guidance's benefit on Transport, consistent with the three-regime view in Section 4.2: when the predicted future state is far from the data manifold, the gradient of distance to distant phase targets pulls toward unreliable targets, washing out the gains from cases where guidance is genuinely useful.

Table 2: Test-time guidance ablations on Transport, evaluated over 950 rollouts (19 starting seeds $\times$ 50 rollouts each). Base policy (no guidance): $0.4664 \pm 0.0140$. Best result in each sub-table in **bold**.

| # Phase targets $M$ | Success Rate |
|---|---|
| 1 | $0.4663 \pm 0.0148$ |
| 10 | $0.4653 \pm 0.0164$ |
| 50 | $\mathbf{0.5358} \pm 0.0169$ |
| 100 | $0.5126 \pm 0.0131$ |
| 150 | $0.4758 \pm 0.0156$ |

(a) Number of phase targets $M$ per phase. Other hyperparameters fixed at $N_p = 4$, $k = 40$, $\tau = 0.7$, thresholds at 50/90.

| Gating strategy | Success Rate |
|---|---|
| No guidance | $0.4664 \pm 0.0140$ |
| Lower only ($\ell_{\text{low}}$=50%) | $0.4800 \pm 0.0170$ |
| Upper only ($\ell_{\text{high}}$=80%) | $0.4947 \pm 0.0138$ |
| Both ($\ell_{\text{low}}$=50%, $\ell_{\text{high}}$=80%) | $\mathbf{0.5105} \pm 0.0131$ |

(b) Effect of guidance gating thresholds. With $M = 50$, $N_p = 4$.

**Guidance Target: Clean Action vs. Noisy Iterate** Section 4.2 argues that the gradient guidance should be computed with respect to the estimated clean action $\hat{A}_t^0$ (the MPGD-style choice [10]) rather than the noisy iterate $A_t^k$ (the formulation used by prior diffusion-policy guidance work [8, 9]), because our dynamics model $d_\phi$ is trained on clean actions and backpropagating through it with noisy inputs violates its training distribution. Table 3 reports the comparison on Transport, averaged over 2,000 rollouts (40 starting seeds $\times$ 50 rollouts each) with all other guidance hyperparameters held fixed. Clean-action guidance outperforms noisy-iterate guidance on average ($0.5135$ vs. $0.4890$, a gap of $\approx 2$ SE), with the direction consistent across seeds and consistent with the MPGD argument: matching the dynamics model's input distribution is what makes its gradient signal informative.

Table 3: Effect of the guidance target on Transport: differentiating through the dynamics model with respect to the estimated clean action $\hat{A}_t^0$ (ours, MPGD-style) vs. the noisy iterate $A_t^k$ (prior diffusion-policy guidance work). Both share the same phase targets, thresholds, and guidance scale; only the gradient target differs. Mean $\pm$ standard error of the mean over 2,000 rollouts (40 starting seeds $\times$ 50 rollouts each).

| Guidance target | # Seeds | Success Rate |
|---|---|---|
| Noisy iterate $A_t^k$ [8, 9] | 40 | $0.4890 \pm 0.0118$ |
| Clean action $\hat{A}_t^0$ (MPGD-style, ours) [10] | 40 | $\mathbf{0.5135} \pm 0.0115$ |

### C.1 Guidance vs. Additional Rollouts

A natural concern is that ReGuide's gains come from extra data rather than guidance specifically. We test this at matched rollout count in both absorption modes of Section 4.3.

**ReGuide-FS.** Appendix Tables 6 and 7 cover Can, Square, and Transport (Tool Hang was not run for this comparison). At every dataset size and on every task, ReGuide-FS trained on demonstrations plus *guided* rollouts outperforms the same procedure on the same number of unguided base-policy rollouts—by 7–10 points on Can and Square and $\approx 7$ points on Transport at the best dataset size.

**ReGuide-FT.** Table 4 reports the comparison on Can under ReGuide-FT: guided rollouts win at every count, with gains of 1.8–4.8 points. Square and Transport versions are in progress.

Table 4: ReGuide-FT on Can with guided vs. base-policy rollouts at matched rollout counts. Both conditions use $\rho = 0.8$. Guided rollouts win at every count.

| # Added Rollouts | Base-policy rollouts | Guided rollouts |
|---|---|---|
| 0 | $0.4924 \pm 0.0101$ | — |
| 8 | $0.5256 \pm 0.0111$ | $\mathbf{0.5436} \pm 0.0096$ |
| 15 | $0.5664 \pm 0.0093$ | $\mathbf{0.5932} \pm 0.0098$ |
| 23 | $0.5376 \pm 0.0101$ | $\mathbf{0.5852} \pm 0.0095$ |

Together, the two modes rule out the "just more data" explanation: at matched volume, guidance contributes distinct value, presumably by covering the drifted-but-recoverable region that the base policy reaches but cannot exit alone.

### C.2 ReGuide-FT vs. ReGuide-FS

The two variants of Section 4.3 play different roles: ReGuide-FS crosses the capacity gap between the base policy and a stronger initialization, while ReGuide-FT refines an already-competent policy. When the loop iterates, that role assignment leaves an engineering question: at each new iteration, should we re-spend compute on another ReGuide-FS run over the accumulated data, or just add another ReGuide-FT round?

Table 5 compares both options on Can at matched rollout counts. Starting from the same ReGuide-FS checkpoint and a newly collected batch of guided rollouts $\mathcal{D}_2^g$, we either (a) run ReGuide-FS on $\mathcal{D}_{\text{demo}} \cup \mathcal{D}_1^g \cup \mathcal{D}_2^g$, or (b) run ReGuide-FT on the merged buffer. The two methods produce comparable final performance: ReGuide-FT slightly ahead at 8 rollouts, ReGuide-FS slightly ahead at 15. The gap in either direction is within one SE.

Table 5: ReGuide-FT vs. ReGuide-FS on Can at matched rollout counts, both starting from the same ReGuide-FS checkpoint (success rate $0.6764$, row 0) and seeing the same newly collected guided rollouts; ReGuide-FT uses $\rho = 0.8$. Performance is statistically comparable; ReGuide-FT wins on compute.

| # Added Rollouts | ReGuide-FT ($\rho = 0.8$) | ReGuide-FS |
|---|---|---|
| 0 | $0.6764 \pm 0.0089$ | — |
| 8 | $\mathbf{0.7208} \pm 0.0086$ | $0.6878 \pm 0.0078$ |
| 15 | $0.6916 \pm 0.0081$ | $\mathbf{0.7088} \pm 0.0087$ |

Given comparable performance, ReGuide-FT is the better engineering choice for iterations beyond the composition step: it converges in far fewer steps from an already-competent policy, while ReGuide-FS repeats most of the original training run. This is what makes the iterative loop practical—each additional round is a ReGuide-FT update, not a ReGuide-FS run.

### C.3 ReGuide-FS per Task

Per-task data for ReGuide-FS: guided vs. base-policy rollouts at matched dataset sizes.

Table 6: Success rates comparing guided vs. base-policy rollouts at matched dataset sizes under ReGuide-FS (Can and Square). Rollouts are merged with the demonstration data and used to train a fresh policy. Guided rollouts outperform base-policy rollouts at every dataset size. Best result per task in **bold**.

| # Rollouts | Base-policy | Guided |
|---|---|---|
| 0 | $0.4924 \pm 0.0101$ | — |
| 8 | $0.5086 \pm 0.0107$ | $0.5324 \pm 0.0100$ |
| 15 | $0.5756 \pm 0.0094$ | $\mathbf{0.6764} \pm 0.0089$ |
| 23 | $0.5528 \pm 0.0093$ | $0.6264 \pm 0.0094$ |

(a) Can.

| # Rollouts | Base-policy | Guided |
|---|---|---|
| 0 | $0.5632 \pm 0.0091$ | — |
| 15 | $0.5628 \pm 0.0101$ | $0.6908 \pm 0.0096$ |
| 30 | $0.6488 \pm 0.0083$ | $\mathbf{0.7144} \pm 0.0101$ |
| 45 | $0.6476 \pm 0.0094$ | $0.6888 \pm 0.0075$ |

(b) Square.

Table 7: Success rates on Transport comparing guided vs. base-policy rollouts at matched dataset sizes under ReGuide-FS. Best result in **bold**.

| # Added Rollouts | Base-policy | Guided |
|---|---|---|
| 0 | $0.4664 \pm 0.0086$ | — |
| 5 | $0.5436 \pm 0.0114$ | $0.6324 \pm 0.0104$ |
| 10 | $0.5924 \pm 0.0092$ | $\mathbf{0.6820} \pm 0.0114$ |
| 15 | $0.6080 \pm 0.0096$ | $0.6544 \pm 0.0096$ |

### C.4 Composition (ReGuide-FT on top of ReGuide-FS) Across Tasks

Per-task detail for the composition: ReGuide-FT starting from the best ReGuide-FS checkpoint, across all four Robomimic tasks.

Table 8: ReGuide-FT applied across tasks starting from the best ReGuide-FS checkpoint (the composition). "Best Scratch" column gives the starting ReGuide-FS policy. Best result per task in **bold**.

| Task | Best ReGuide-FS | # Rollouts | Success Rate |
|---|---|---|---|
| Can | $0.6764 \pm 0.0089$ | 8 | $0.7208 \pm 0.0086$ |
| | | 15 | $0.6916 \pm 0.0081$ |
| | | 23 | $\mathbf{0.7228} \pm 0.0083$ |
| Square | $0.7144 \pm 0.0101$ | 15 | $0.6528 \pm 0.0110$ |
| | | 30 | $0.6596 \pm 0.0104$ |
| | | 45 | $\mathbf{0.7336} \pm 0.0091$ |
| Transport | $0.682 \pm 0.0114$ | 5 | $0.5896 \pm 0.0107$ |
| | | 10 | $\mathbf{0.7056} \pm 0.0086$ |
| | | 15 | $0.6664 \pm 0.0106$ |
| Tool Hang | $0.1656 \pm 0.0078$ | 20 | $\mathbf{0.2332} \pm 0.0106$ |
| | | 40 | $0.1776 \pm 0.0071$ |
| | | 60 | $0.1976 \pm 0.0075$ |
| | | 80 | $0.1836 \pm 0.0076$ |

### C.5 Iterating ReGuide-FT from the Base Policy

Per-task detail across all four tasks for both ReGuide-FT iterations. This is the data underlying Figure 4 and the iteration discussion in the main paper.

Table 9: Iterative ReGuide-FT from the base policy across tasks: (*top*) Iteration 1, starting from the base diffusion policy; (*bottom*) Iteration 2, starting from the best policy of Iteration 1. **# Total Guided Rollouts** counts the cumulative guided rollouts in the training set across both iterations; Iteration 2 rows include the Iteration 1 budget that produced its starting policy (Can: 15, Square: 30, Transport: 10, Tool Hang: 20). Best result per task in **bold**.

| Task | Base | # Total Guided Rollouts | Success Rate |
|---|---|---|---|
| Can | $0.4924 \pm 0.0101$ | 8 | $0.5436 \pm 0.0096$ |
| | | 15 | $\mathbf{0.5932} \pm 0.0098$ |
| | | 23 | $0.5852 \pm 0.0095$ |
| Square | $0.5632 \pm 0.0091$ | 15 | $0.5812 \pm 0.0107$ |
| | | 30 | $\mathbf{0.6592} \pm 0.0095$ |
| | | 45 | $0.6500 \pm 0.0101$ |
| Transport | $0.4664 \pm 0.0086$ | 5 | $0.5692 \pm 0.0096$ |
| | | 10 | $\mathbf{0.6456} \pm 0.0110$ |
| | | 15 | $0.5904 \pm 0.0092$ |
| Tool Hang | $0.0332 \pm 0.003$ | 20 | $\mathbf{0.1592} \pm 0.007$ |
| | | 40 | $0.1568 \pm 0.006$ |

(a) Iteration 1 (from base policy).

| Task | Iter. 1 Best | # Total Guided Rollouts | Success Rate |
|---|---|---|---|
| Can | $0.5932 \pm 0.0098$ | 23 | $0.6012 \pm 0.0082$ |
| | | 30 | $0.5592 \pm 0.0087$ |
| | | 38 | $0.5940 \pm 0.0084$ |
| | | 45 | $0.5836 \pm 0.0086$ |
| | | 60 | $\mathbf{0.6236} \pm 0.0090$ |
| | | 75 | $0.6172 \pm 0.0097$ |
| | | 90 | $0.6036 \pm 0.0092$ |
| Square | $0.6592 \pm 0.0095$ | 45 | $0.6124 \pm 0.0107$ |
| | | 60 | $0.6584 \pm 0.0107$ |
| | | 75 | $0.6612 \pm 0.0088$ |
| | | 120 | $0.7068 \pm 0.0088$ |
| | | 150 | $\mathbf{0.7120} \pm 0.0085$ |
| | | 180 | $0.6580 \pm 0.0098$ |
| Transport | $0.6456 \pm 0.0110$ | 15 | $0.6644 \pm 0.0082$ |
| | | 25 | $0.5656 \pm 0.0115$ |
| | | 40 | $0.6676 \pm 0.0081$ |
| | | 50 | $0.6776 \pm 0.0089$ |
| | | 60 | $\mathbf{0.6948} \pm 0.0094$ |
| Tool Hang | $0.1592 \pm 0.007$ | 40 | $\mathbf{0.2404} \pm 0.008$ |
| | | 60 | $0.2248 \pm 0.007$ |
| | | 80 | $0.1980 \pm 0.008$ |
| | | 100 | $0.2104 \pm 0.008$ |

(b) Iteration 2 (from best of Iteration 1).

### C.6 Buffer-Share Ratio Ablation

Ablation over the buffer-share ratio $\rho$ used in ReGuide-FT (Section 4.3).

Table 10: Ablation over buffer-share ratio $\rho$ on Can with 15 added guided rollouts during continual learning, evaluated over 950 rollouts (19 starting seeds $\times$ 50 rollouts each). Share denotes the percentage of each minibatch drawn from the newly collected guided rollouts.

| $\rho$ | Success Rate |
|---|---|
| 0.6 | $0.5800 \pm 0.0177$ |
| 0.7 | $\mathbf{0.6008} \pm 0.0159$ |
| 0.8 | $0.5932 \pm 0.0159$ |
| 0.9 | $0.5680 \pm 0.0146$ |
| 1 | $0.5608 \pm 0.0178$ |

# D Setup Details

This section collects per-task implementation details deferred from the Setup paragraph in Section 5.

**Rollout-count sweeps.** The number of guided rollouts added per iteration is set as a small multiple of the demonstration count for each task: approximately $0.5\times$, $1\times$, and $1.5\times$ in Iteration 1 for Can, Square, and Transport, with higher multiples in Iteration 2 to extend the iteration-2 sweep (see Table 9). Tool Hang uses smaller relative multipliers ($0.25\times$ and $0.5\times$) to keep retraining cost manageable on the largest base policy.

**Buffer-share ratio.** For both ReGuide-FT and the FT step of the composition we use a merged buffer with $\rho \in [0.7, 0.8]$, with $\rho$ chosen best-per-task and the newly collected guided rollouts weighted more heavily in each minibatch. The ratio is a stability/plasticity knob: a higher proportion of original demonstration data preserves learned behavior and keeps the update stable but limits how much new behavior the policy can absorb, while a higher proportion of new rollouts accelerates learning of new behavior at the risk of distributional shift and forgetting. The choice is mild within our chosen range—the ablation in Table 10 shows performance is roughly flat on Can.

**Supervision assumption.** The rollout-collection step filters for successful trajectories, so the pipeline assumes access to a binary task-success signal at trajectory completion. This is the same assumption made implicitly by any self-improvement scheme that selects among self-collected data, and is strictly weaker than DAgger-style per-state expert relabeling: success-filtering needs one bit per trajectory, not an action label at every visited state. In Robomimic this signal is provided by the simulator; in deployment it would come from a task-specific success classifier, a sparse reward, or human verification at episode end.

**Acknowledgments**

If a paper is accepted, the final camera-ready version will (and probably should) include acknowledgments. All acknowledgments go at the end of the paper, including thanks to reviewers who gave useful comments, to colleagues who contributed to the ideas, and to funding agencies and corporate sponsors that provided financial support.